Overall Explanation

The Purpose of this program is to build a simple Company Employee Access System using the basic concept of (**OPP**) in Java .

The main concepts used in this program are **encapsulation**, **inheritance**, **constructors**, **method overriding**, **and** **static variables**.

First, an **`Employee`** class is created as the parent class. It contains three private fields: `employeeId`, `name`, and `dailyAccessHours`. The employee ID is generated automatically using a private static counter, so every new employee receives a unique ID. The normal daily access time for an employee is set to 8 hours.

Getters are provided to access the private fields, but no setter is provided for `employeeId` because the ID should not be changed manually.

Next, the `Manager` class is created by **extending** the `Employee` class. This demonstrates **inheritance**, because a Manager is also an Employee. In addition to the properties inherited from Employee, a Manager has a private `teamSize` field. The `getDailyAccessHours()` method is overridden so that a manager gets additional access hours based on the team size. The formula used is `8 + (teamSize / 5)`. For example, if the team size is 12, the manager receives **`8 + 2 = 10`** hours.

The **`Intern` class** also extends the `Employee` class. It has an additional private field called `month`. Its `getDailyAccessHours()` method is overridden according to the internship month. Interns receive 4 hours during months 1 to 3 and 6 hours during months 4 to 6.

Finally, the `Main` class is used to test the complete program. It creates one normal Employee, one Manager, and one Intern. The `describe()` method is called for each object to display their information, and their daily access hours are also printed. Another Employee is created with a blank name to demonstrate the validation feature.

Overall, this program demonstrates how OOP can be used to represent different types of employees while keeping the code organized and reusable. The `Employee` class contains the common properties and behavior, while `Manager` and `Intern` extend it and provide their own specific behavior through method overriding.

The project is divided into four Java files:

EmployeeAccessSystem/

Employee.java → Parent class with common employee details

Manager.java → Child class for manager-specific features

Intern.java → Child class for intern-specific features

Main.java → Creates objects and runs the program